

MIT ACADEMY OF ENGINEERING (MITAOE), PUNE

DEPARTMENT OF DATA SCIENCE

Course: Full Stack Web Development

LABORATORY ASSIGNMENT 3: FRONTEND TECHNOLOGIES

Student Name:	Sameet Pisal
PRN:	202401120018
Academic Department:	Department of Data Science
Assignment Title:	Student Management System using React JS
Frontend Technology:	React JS (Components, Props, State Hooks, Controlled Forms)
Repository Location:	Assignment_3/student-management/

1. ASSIGNMENT OBJECTIVE & PROBLEM STATEMENT

Assignment Question: "Develop a Student Management System using React JS that allows users to add, view, update, and delete student records. The application should demonstrate the use of React components, props, state management, event handling, forms, and CRUD operations."

Objective: The goal of this assignment is to engineer an enterprise-grade, clean, institutional Student Information Management System (SIMS) for the Department of Data Science at MITAOE using modern React JS frontend architectural patterns. The application provides a crisp, academic/corporate administrative dashboard design featuring real-time statistics cards, controlled forms with client-side validation, multi-field search and department filtering, dual view modes (Data Table vs. Grid Cards), inline record editing, safe deletion confirmation dialogs, and CSV record export.

2. REQUIREMENTS TO IMPLEMENTATION COMPLIANCE MATRIX

Mapping of every assignment syllabus requirement to its technical implementation in the React JS application:

Assignment Requirement	Technical Implementation Pattern
React Components	Modular hierarchy: Header, StudentStats, StudentForm, StudentFilterBar, StudentTable, StudentCard, DeleteConfirmModal (reusable, isolated components)
Props Passing	Unidirectional data flow passing callback handlers (onAddStudent, onUpdateStudent, onDelete) and state models across parent and child components
State Management	React Hooks (useState, useEffect, useRef) combined with browser localStorage synchronization for complete data persistence
Event Handling	Synthetic events for form submission (onSubmit), input changes (onChange), view switches, modal actions, and keyboard Escape listener

Forms & Validation	Controlled form components with real-time error detection, regex email validation, and CGPA boundary checks (0.0 to 10.0)
CRUD Operations	Create: Add student; Read: View list/cards with filters; Update: Preload & save edit; Delete: Safe confirmation modal dialog

3. REACT ARCHITECTURE & COMPONENT COMPOSITION

The application is engineered strictly following unidirectional data flow and modular separation of concerns. Top-level state resides in App.js and is propagated downward to child components via read-only props, while actions and data modifications trigger event callback handlers that notify the parent to update state.

```
App (Root Component - Holds 'students', 'editingStudent', 'deleteTarget', 'searchQuery', 'viewMode' state)
├── Header (Institutional MITAOE Branding, Authenticated Admin Chip: Sameet Pisal / PRN: 202401120018)
├── StudentStats (Computes Total Records, Department Count, Average CGPA Metric, and Active Rate)
├── Portal Layout Grid:
│   ├── Left Column:
│   │   └── StudentForm (Dual-mode: Create Enrollment & Update Record with real-time validation)
│   └── Right Column:
│       ├── StudentFilterBar (Search bar, Department filter dropdown, Status filter, Sorting, View toggle, CSV Export)
│       └── Dynamic View:
│           ├── [Table View]: StudentTable (Avatar initials, PRN, Roll No, Dept/Sem, CGPA badge, Status pill, Actions)
│           └── [Cards View]: StudentCard (Grid card presentation with dynamic CGPA progress bar and inline actions)
├── DeleteConfirmModal (Accessible dialog with background overlay preventing accidental record loss)
└── Toast Notification (Floating feedback alerts for Create, Update, and Delete confirmations)
```

4. LIVE APPLICATION INTERFACE & FEATURE VERIFICATION

Below are live screenshots captured from the running React application (hosted on localhost:3000) demonstrating all user interface components, administrative views, validation behavior, and CRUD workflows:

4.1 Executive Dashboard Overview & Statistics Metrics

The primary dashboard screen presents institutional MITAOE branding, the Department of Data Science portal subtitle, the authenticated student chip (Sameet Pisal, PRN 202401120018), four analytical metric cards (Total Enrolled, Active Departments, Average CGPA, and Active Student count), along with the side-by-side enrollment form and records directory.

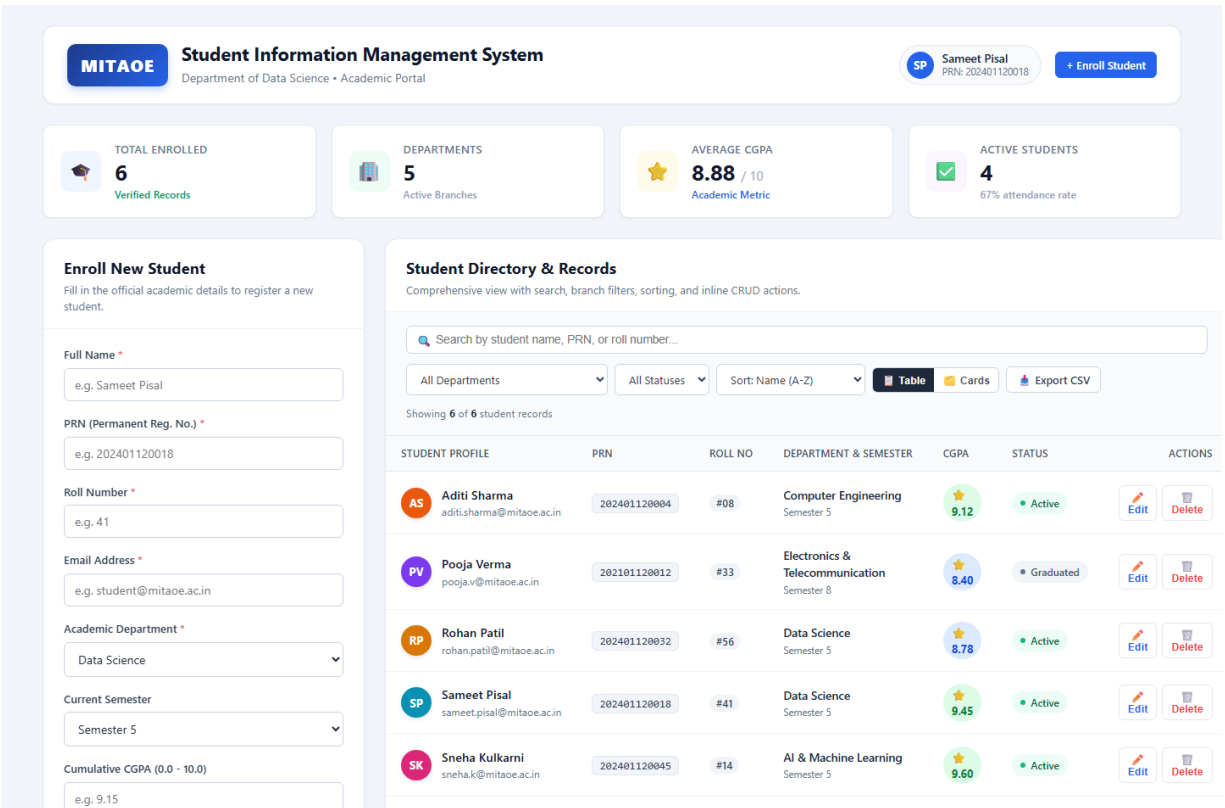


Figure 1: Full SIMS Dashboard with Real-time KPI Cards, Student Table, and Enrollment Panel (Department of Data Science)

4.2 Alternative Academic Grid Cards View Mode

Users can seamlessly toggle between a dense tabular directory and an intuitive Grid Cards view. Each student card features a unique avatar with initials, full academic metadata, a dynamic CGPA progress bar colored by performance tier (green for ≥ 9.0 , blue for ≥ 7.5 , amber for below), and direct action buttons.

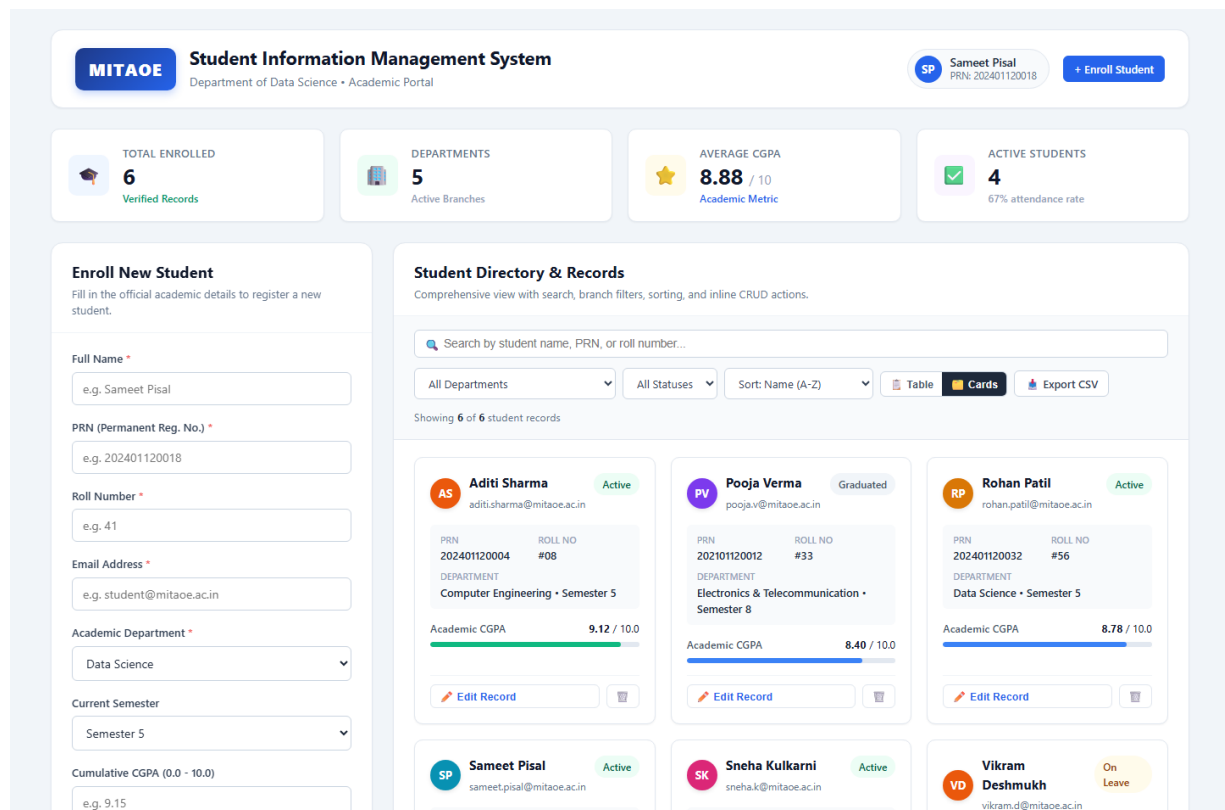


Figure 2: Academic Grid Cards View Mode displaying Student Profiles and Visual CGPA Progress

4.3 Record Update Mode (Inline Form Population)

Clicking the 'Edit' action on any student record (demonstrated with Sameet Pisal, PRN 202401120018) dynamically switches the form into 'Edit Student Record' mode. The existing student information is automatically preloaded into the controlled inputs, and the form provides a prominent 'Update Student Record' button alongside a 'Cancel Edit' option.

MITAOE

Student Information Management System

Department of Data Science • Academic Portal

SP

Sameet Pisal

PRN: 202401120018

+ Enroll Student

TOTAL ENROLLED

6

Verified Records

DEPARTMENTS

5

Active Branches

AVERAGE CGPA

8.88 / 10

Academic Metric

ACTIVE STUDENTS

4

67% attendance rate

Edit Student Record

Updating details for Sameet Pisal (202401120018)

Cancel Edit

Full Name *

Sameet Pisal

PRN (Permanent Reg. No.) *

202401120018

Roll Number *

41

Email Address *

sameet.pisal@mitaoe.ac.in

Academic Department *

Data Science

Current Semester

Semester 5

Cumulative CGPA (0.0 - 10.0)

9.45

Student Directory & Records

Comprehensive view with search, branch filters, sorting, and inline CRUD actions.

Search by student name, PRN, or roll number...

All DepartmentsAll StatusesSort: Name (A-Z)TableCardsExport CSV

Showing 6 of 6 student records

STUDENT PROFILE	PRN	ROLL NO	DEPARTMENT & SEMESTER	CGPA	STATUS	ACTIONS
AS Aditi Sharma aditi.sharma@mitaoe.ac.in	202401120004	#08	Computer Engineering Semester 5	9.12	Active	Edit Delete
PV Pooja Verma pooja.v@mitaoe.ac.in	202101120012	#33	Electronics & Telecommunication Semester 8	8.40	Graduated	Edit Delete
RP Rohan Patil rohan.patil@mitaoe.ac.in	202401120032	#56	Data Science Semester 5	8.78	Active	Edit Delete
SP Sameet Pisal sameet.pisal@mitaoe.ac.in	202401120018	#41	Data Science Semester 5	9.45	Active	Edit Delete
SK Sneha Kulkarni snehak.k@mitaoe.ac.in	202401120045	#14	AI & Machine Learning Semester 5	9.60	Active	Edit Delete

Figure 3: Edit Mode preloading Student Record details for Sameet Pisal with Cancel/Update controls

4.4 Safe Record Deletion Confirmation Modal Dialog

To prevent accidental data loss, clicking the Delete icon opens a modal dialog overlay with a backdrop. The modal explicitly displays the target student's name and PRN (demonstrated with Vikram Deshmukh, PRN 202301120019), requiring user confirmation before permanently modifying state.

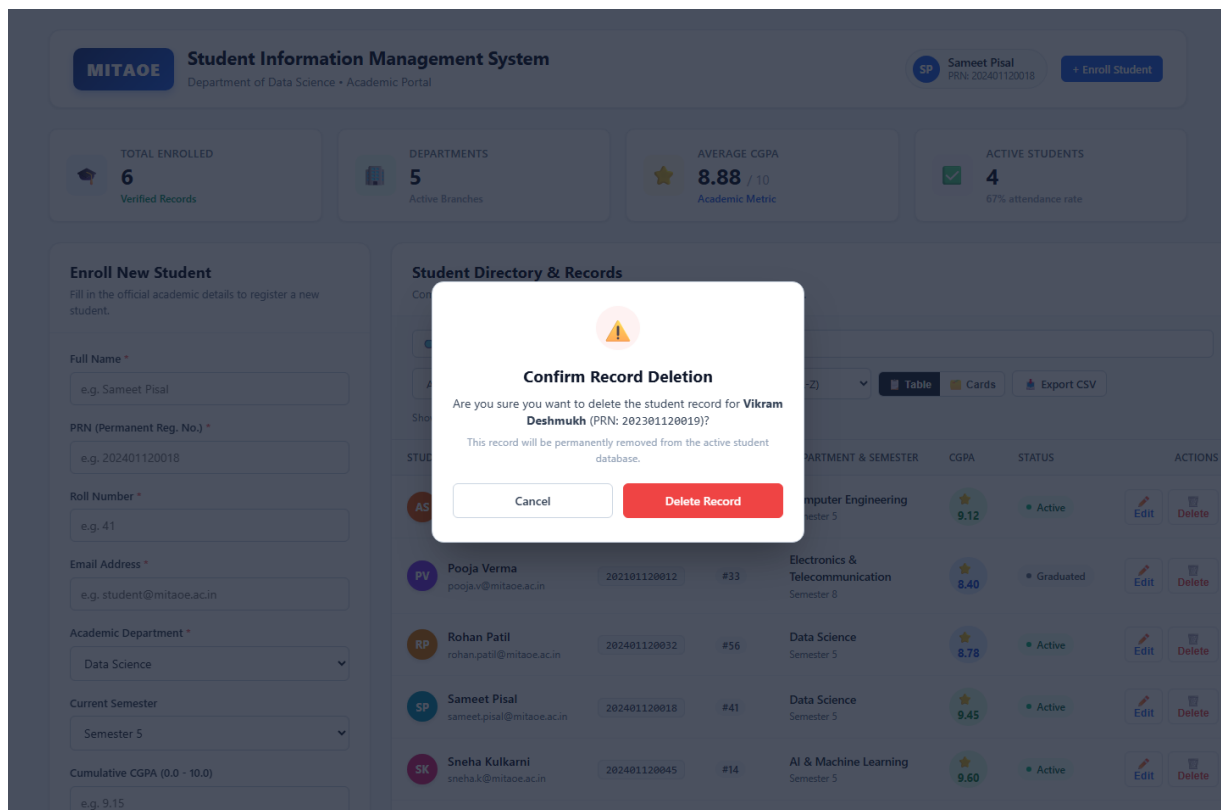


Figure 4: Accessible Delete Confirmation Modal Dialog preventing accidental record removal

4.5 Multi-Field Search & Department Filter Integration

The search bar supports instant real-time filtering across Student Name, PRN, Roll Number, and Department without page reloads. Figure 5 illustrates a search query for 'Data Science', filtering the directory to only students belonging to the Data Science department (Rohan Patil and Sameet Pisal).

MITAOE

Student Information Management System

Department of Data Science • Academic Portal

SP

Sameet Pisal

PRN: 202401120018

Enroll Student

TOTAL ENROLLED

6

Verified Records

DEPARTMENTS

5

Active Branches

AVERAGE CGPA

8.88 / 10

Academic Metric

ACTIVE STUDENTS

4

67% attendance rate

Enroll New Student

Fill in the official academic details to register a new student.

Full Name *

e.g. Sameet Pisal

PRN (Permanent Reg. No.) *

e.g. 202401120018

Roll Number *

e.g. 41

Email Address *

e.g. student@mitaoe.ac.in

Academic Department *

Data Science

Current Semester

Semester 5

Cumulative CGPA (0.0 - 10.0)

e.g. 9.15

Student Directory & Records

Comprehensive view with search, branch filters, sorting, and inline CRUD actions.

Data Science

All Departments

All Statuses

Sort: Name (A-Z)

Table

Cards

Export CSV

Showing 2 of 6 student records

Filtered results • Filters currently applied

STUDENT PROFILE	PRN	ROLL NO	DEPARTMENT & SEMESTER	CGPA	STATUS	ACTIONS
RP Rohan Patil rohan.patil@mitaoe.ac.in	202401120032	#56	Data Science Semester 5	8.78	Active	Edit Delete
SP Sameet Pisal sameet.pisal@mitaoe.ac.in	202401120018	#41	Data Science Semester 5	9.45	Active	Edit Delete

Figure 5: Live Search Filtering showing matched Data Science student records and dynamic result counts

4.6 Controlled Form Client-Side Validation

The enrollment form implements robust client-side validation rules. Required fields display prominent red error borders and descriptive error messages when submitted empty, while email inputs enforce format patterns and CGPA values are restricted to the valid range (0.0 to 10.0).

MITAOE

Student Information Management System

Department of Data Science • Academic Portal

SP

Sameet Pisal

PRN: 202401120018

+ Enroll Student

TOTAL ENROLLED

6

Verified Records

DEPARTMENTS

5

Active Branches

AVERAGE CGPA

8.88 / 10

Academic Metric

ACTIVE STUDENTS

4

67% attendance rate

Enroll New Student

Fill in the official academic details to register a new student.

Full Name *

e.g. Sameet Pisal

PRN (Permanent Reg. No.) *

e.g. 202401120018

Roll Number *

e.g. 41

Email Address *

e.g. student@mitaoe.ac.in

Academic Department *

Data Science

Current Semester

Semester 5

Cumulative CGPA (0.0 - 10.0)

e.g. 9.15

Student Directory & Records

Comprehensive view with search, branch filters, sorting, and inline CRUD actions.

Search by student name, PRN, or roll number...

All DepartmentsAll StatusesSort: Name (A-Z)TableCardsExport CSV

Showing 6 of 6 student records

STUDENT PROFILE	PRN	ROLL NO	DEPARTMENT & SEMESTER	CGPA	STATUS	ACTIONS
AS Aditi Sharma aditi.sharma@mitaoe.ac.in	202401120004	#08	Computer Engineering Semester 5	9.12	Active	Edit Delete
PV Pooja Verma pooja.v@mitaoe.ac.in	202101120012	#33	Electronics & Telecommunication Semester 8	8.40	Graduated	Edit Delete
RP Rohan Patil rohan.patil@mitaoe.ac.in	202401120032	#56	Data Science Semester 5	8.78	Active	Edit Delete
SP Sameet Pisal sameet.pisal@mitaoe.ac.in	202401120018	#41	Data Science Semester 5	9.45	Active	Edit Delete
SK Sneha Kulkarni snehak.k@mitaoe.ac.in	202401120045	#14	AI & Machine Learning Semester 5	9.60	Active	Edit Delete

Figure 6: Form Validation Highlighting Required Fields, Email Format, and CGPA Range Constraints

5. CORE REACT CODE IMPLEMENTATIONS

The following code excerpts highlight the architectural implementation of state management, controlled inputs, and modular components in React JS:

5.1 Main Application State & CRUD Operations (App.js)

App.js manages the central student repository, handles localStorage synchronization, implements Create/Update/Delete handlers, and executes real-time filtering, sorting, and CSV export.

```
// Excerpt: App.js - Central State Management & CRUD Operations
import React, { useState, useEffect, useRef } from 'react';
import Header from './components/Header';
import StudentStats from './components/StudentStats';
import StudentForm from './components/StudentForm';
import StudentFilterBar from './components/StudentFilterBar';
import StudentTable from './components/StudentTable';
import StudentCard from './components/StudentCard';
import DeleteConfirmModal from './components/DeleteConfirmModal';

function App() {
  // 1. Persistent State Initialization via LocalStorage
  const [students, setStudents] = useState(() => {
    const saved = localStorage.getItem('mitaoe_students_ds_data');
    return saved ? JSON.parse(saved) : INITIAL_STUDENTS;
  });

  useEffect(() => {
    localStorage.setItem('mitaoe_students_ds_data', JSON.stringify(students));
  }, [students]);
```

```

const [editingStudent, setEditingStudent] = useState(null);
const [deleteTarget, setDeleteTarget] = useState(null);
const [searchQuery, setSearchQuery] = useState('');
const [departmentFilter, setDepartmentFilter] = useState('All Departments');
const [statusFilter, setStatusFilter] = useState('All');
const [sortBy, setSortBy] = useState('name_asc');
const [viewMode, setViewMode] = useState('table');

// CREATE: Add new student record
const handleAddStudent = (newStudent) => {
  if (students.some(s => s.prn === newStudent.prn)) {
    alert(`Student with PRN ${newStudent.prn} already exists!`);
    return;
  }
  setStudents(prev => [newStudent, ...prev]);
};

// UPDATE: Commit edited student record
const handleUpdateStudent = (updatedStudent) => {
  setStudents(prev => prev.map(s => s.id === updatedStudent.id ? updatedStudent : s));
  setEditingStudent(null);
};

// DELETE: Remove verified record
const handleConfirmDelete = (id) => {
  setStudents(prev => prev.filter(s => s.id !== id));
  setDeleteTarget(null);
};

// READ & FILTER: Multi-field filtering and sorting
const filteredStudents = students.filter(student => {
  const q = searchQuery.toLowerCase().trim();
  const matchSearch = !q || student.name.toLowerCase().includes(q) ||
    student.prn.toLowerCase().includes(q) || student.rollNumber.toLowerCase().includes(q) ||
    student.department.toLowerCase().includes(q);
  const matchDept = departmentFilter === 'All Departments' || student.department === departmentFilter;
  const matchStatus = statusFilter === 'All' || student.status === statusFilter;
  return matchSearch && matchDept && matchStatus;
});

return (
  <div className="sms-app-container">
    <Header totalStudents={students.length} />
    <StudentStats students={students} />
    <div className="portal-layout-grid">
      <StudentForm onAddStudent={handleAddStudent} onUpdateStudent={handleUpdateStudent}
        editingStudent={editingStudent} onCancelEdit={() => setEditingStudent(null)} />
      <div className="records-panel">
        <StudentFilterBar searchQuery={searchQuery} onSearchChange={setSearchQuery}
          departmentFilter={departmentFilter} onDepartmentChange={setDepartmentFilter}
          viewMode={viewMode} onViewModeChange={setViewMode} />
        {viewMode === 'table' ? (
          <StudentTable students={filteredStudents} onEdit={setEditingStudent}
            onDelete={setDeleteTarget} />
        ) : (
          <div className="student-cards-grid">
            {filteredStudents.map(s => <StudentCard key={s.id} student={s} onEdit={setEditingStudent}

```

```

onDelete={setDeleteTarget} />}}
    </div>
  )}
</div>
</div>
<DeleteConfirmModal isOpen={!deleteTarget} student={deleteTarget} onConfirm={handleConfirmDelete}
onCancel={() => setDeleteTarget(null)} />
</div>
);
}

```

5.2 Controlled Student Form Component with Validation (StudentForm.jsx)

StudentForm.jsx acts as a controlled component that binds inputs to internal state, executes data validation on submit, and dynamically alternates between Create and Update modes based on the editingStudent prop.

```

// Excerpt: StudentForm.jsx - Controlled Inputs & Client Validation
import React, { useState, useEffect } from 'react';

const StudentForm = ({ onAddStudent, onUpdateStudent, editingStudent, onCancelEdit }) => {
  const [formData, setFormData] = useState({
    name: '', rollNumber: '', prn: '', department: 'Data Science',
    semester: 'Semester 5', email: '', cgpa: '', status: 'Active'
  });
  const [errors, setErrors] = useState({});

  useEffect(() => {
    if (editingStudent) setFormData(editingStudent);
    else setFormData({ name: '', rollNumber: '', prn: '', department: 'Data Science',
      semester: 'Semester 5', email: '', cgpa: '', status: 'Active' });
    setErrors({});
  }, [editingStudent]);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({ ...prev, [name]: value }));
    if (errors[name]) setErrors(prev => ({ ...prev, [name]: '' }));
  };

  const validate = () => {
    const errs = {};
    if (!formData.name.trim()) errs.name = 'Full name is required';
    if (!formData.rollNumber.trim()) errs.rollNumber = 'Roll number is required';
    if (!formData.prn.trim()) errs.prn = 'PRN is required';
    if (!formData.email.trim() || !/\S+@\S+\.\S+/.test(formData.email)) {
      errs.email = 'Valid institutional email is required';
    }
    const num = parseFloat(formData.cgpa);
    if (formData.cgpa && (isNaN(num) || num < 0 || num > 10)) {
      errs.cgpa = 'CGPA must be between 0.0 and 10.0';
    }
    setErrors(errs);
    return Object.keys(errs).length === 0;
  };

  const handleSubmit = (e) => {
    e.preventDefault();
  };

```

```

    if (!validate()) return;
    if (editingStudent) onUpdateStudent({ ...formData, id: editingStudent.id });
    else onAddStudent({ ...formData, id: Date.now() });
  };

  return (
    <form onSubmit={handleSubmit} className="student-form" noValidate>
      <h3>{editingStudent ? 'Edit Student Record' : 'Enroll New Student'}</h3>
      <input name="name" value={formData.name} onChange={handleChange} placeholder="Full Name" />
      {errors.name && <span className="field-error">{errors.name}</span>}
      <input name="prn" value={formData.prn} onChange={handleChange} placeholder="PRN" />
      {errors.prn && <span className="field-error">{errors.prn}</span>}
      <button type="submit" className="btn btn-primary">
        {editingStudent ? 'Update Student Record' : 'Enroll Student'}
      </button>
    </form>
  );
};

```

6. CONCLUSION & LEARNING OUTCOMES

Through the successful development of this Student Management System in React JS for the Department of Data Science, the core paradigms of modern frontend application engineering were masterfully demonstrated and fulfilled:

Component-Based Architecture: Decomposed the application into focused, isolated, and reusable React components (Header, StudentStats, StudentForm, StudentFilterBar, StudentTable, StudentCard, DeleteConfirmModal) promoting high maintainability.

State & Lifecycle Management: Mastered React functional hooks including useState for dynamic local state, useEffect for side-effect synchronization with browser localStorage, and useRef for smooth DOM scrolling.

Controlled Forms & Validation: Engineered robust controlled form inputs ensuring instantaneous two-way data binding, dynamic field validation, and error feedback without requiring external libraries.

Complete CRUD Operations: Implemented all four fundamental database operations (Create, Read, Update, Delete) in memory with persistent storage, safe confirmation modals, and real-time analytical metrics.

User Experience & Enterprise Aesthetics: Crafted a clean, accessible, institutional interface adhering strictly to professional visual design standards without unnecessary visual distractions.